BMI/CS 771 - Homework 2 Report

Team: Ashvin, Akshat, Shubhangi

Part I - Image Classification (MiniPlaces)

I.1 Custom CNN

The provided SimpleNet is a shallow network with 7×7 convs, no normalization and about 0.37M parameters. We replaced it with `CustomNet` (in `cls_libs/modeling.py`), a small ResNet-style network with about 11.2M parameters. We made three changes to fix the problems with the baseline:

- BatchNorm after every conv. This makes training more stable and lets us use a big learning rate.
- Skip connections. The body is eight residual `BasicBlock`s (two 3×3 convs each). When the stride or channel count changes, the skip path gets a 1×1 projection.
- More depth. There are four stages with two blocks each, widths 64/128/256/512, and stride-2 downsampling at the start of stages 2 to 4.

The stem is a 3×3 stride-2 conv plus max-pool, which takes the 128×128 input down to 32×32. At the end we have global average pooling, dropout (0.2) and a linear classifier. Convs use Kaiming init. The last BatchNorm in each residual branch starts at zero, so every block begins as an identity mapping.

Training. We used the provided pipeline and defaults: SGD (momentum 0.9, weight decay 1e-4), lr 0.1 with one warmup epoch and then cosine decay, batch size 512, fp16, 60 epochs. Augmentation was random crop to 128×128, color jitter and rotation. We trained on a single GPU on CHTC.

Results (MiniPlaces validation set):

	Top-1	**Top-5**
Final checkpoint, `eval_cls.py` (with horizontal-flip averaging)	49.83	78.36
Final epoch, training-time validation (no flip)	49.18	77.51

The first row is from running `eval_cls.py` on `model_final.pth.tar`. It averages the predictions for each image and its horizontal flip. The second row is the validation accuracy logged during training, which has no flip. We use the first row as our final result.

Validation accuracy flattens out over the last ~10 epochs as the cosine schedule decays. Training top-1 (mean of the last five logged batches) is about 66% while validation is about 49%, so the network overfits.

Files: config `results/cls_cnn/cls_miniplaces_cnn.yaml`, logs `results/cls_cnn/tensorboard/`, `config.txt` and `eval_cls_miniplaces_cnn.txt`, final checkpoint `results/cls_cnn/model_final.pth.tar`.

I.2 Comparison: Custom CNN vs. SimpleNet vs. ResNet-18

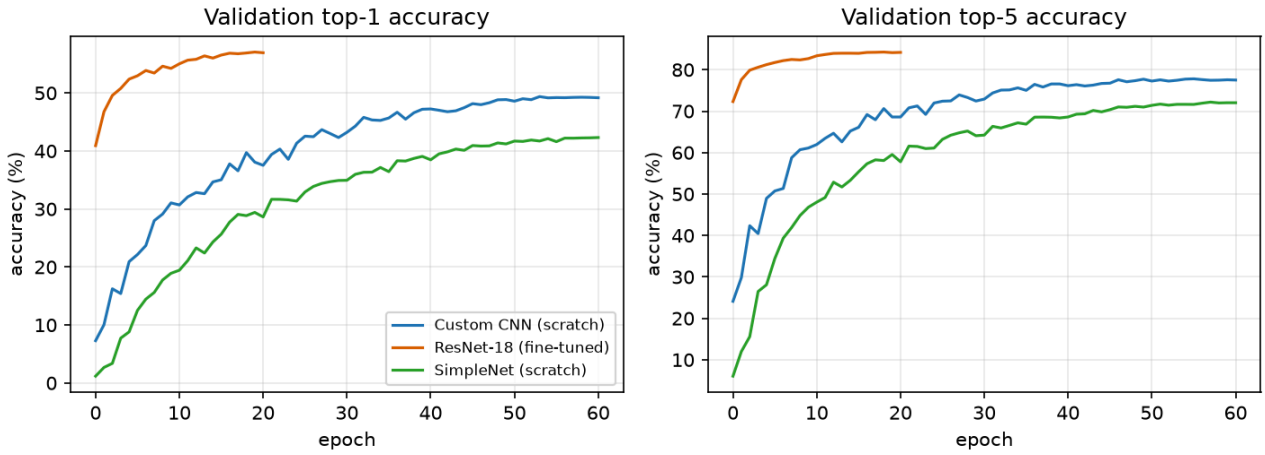


Figure 1: Validation accuracy during training (no flip averaging) for SimpleNet (provided, from scratch, 60 epochs), our custom CNN (from scratch, 60 epochs) and fine-tuned ResNet-18 (20 epochs).

Model	Initialization	Params	Epochs	Top-1	Top-5
SimpleNet (provided)	scratch	~0.37M	60	42.66	72.55
Custom CNN	scratch	~11.2M	60	49.83	78.36
ResNet-18	ImageNet-pretrained, fine-tuned	~11.2M	20	57.68	84.91

We trained SimpleNet with the default config (`results/cls_simplenet/`). It gets 42.66% top-1, which is close to the ~40% given in the assignment. Its training top-1 (43.4%) is about the same as its validation top-1 (42.3%), so it is held back by capacity and not by overfitting. Our CNN is the opposite, it overfits (66% train vs. 49% val).

Our CNN beats SimpleNet by about 7 points top-1 (49.83% vs. 42.66%) and about 6 points top-5. Some of this comes from BatchNorm and skip connections, which make the deeper network trainable. Some comes from just having a lot more parameters. We didn't run an ablation, so we can't say how much each one matters.

Fine-tuned ResNet-18 is the best of the three. It is about 8 points above our CNN (57.68% vs. 49.83% top-1) and 15 points above SimpleNet, even though it trained for a third of the epochs. Its architecture and parameter count are almost the same as ours, so the gap is from ImageNet pretraining and not from the model design. The pretrained network is already at ~41% top-1 after the first epoch. Ours starts near 7% and needs about 25 to 30 epochs to pass SimpleNet's final accuracy.

ResNet-18 fine-tuning used lr 0.01 and the same optimizer, schedule and augmentation as the CNN run. Its config, logs and eval output are in `results/cls_resnet18/`. The SimpleNet config, logs, eval output

and checkpoint are in `results/cls_simplenet/`. The table values come from `eval_cls.py` on the final checkpoints. The curves show training-time validation accuracy, which is about 0.7 points lower because there is no flip averaging.

I.3 Self-Attention and Transformer Block

Multi-Head Self-Attention (MSA)

Self-attention builds each token's output as a weighted sum of the other tokens, where the weights come from how similar the query and key projections are. The input is a sequence of tokens $Z^0 \in \mathbb{R}^{T \times D}$, with $T = H \times W$ and D the embedding size. Three linear layers give the queries, keys and values:

$$Q = Z^0 W_Q, \quad K = Z^0 W_K, \quad V = Z^0 W_V$$

where $W_Q, W_K, W_V \in \mathbb{R}^{D \times D}$. With h heads, the channels are split so each head works on vectors of size $d_k = D/h$. For each head we take scaled dot-product similarities, apply a row-wise softmax, and multiply by V:

$$S = \mathrm{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

The $\frac{1}{\sqrt{d_k}}$ factor keeps the softmax from saturating (and its gradients from vanishing) when the head size is large. The outputs of all heads are concatenated and passed through an output layer $W_O \in \mathbb{R}^{D \times D}$.

In `cls_libs/modeling.py`, `Attention.forward` takes an input of shape `(B, H, W, C)` and produces queries, keys and values of shape `(B * num_heads, H * W, head_dim)`. After the attention and the multiplication by V, we reshape to `(B, num_heads, H, W, head_dim)`, permute to `(B, H, W, num_heads, head_dim)`, flatten back to `(B, H, W, C)`, and apply `self.proj`.

Pre-LN Transformer Block with Local Window Self-Attention

Global self-attention over all T tokens costs $\mathcal{O}(T^2 D + D^2 T)$. This gets expensive fast for large feature maps. Local window attention splits the tokens into non-overlapping $W_w \times W_w$ windows and attends only inside each window. The cost drops to $\mathcal{O}(W_w^2 T D + D^2 T)$, and W_w^2 is much smaller than T.

The block is Pre-LayerNorm with stochastic depth (DropPath):

$$x_1 = x_0 + \mathrm{DropPath}\left(\mathrm{WindowMSA}(\mathrm{LN}_1(x_0))\right)$$

$$x_2 = x_1 + \mathrm{DropPath}\left(\mathrm{MLP}(\mathrm{LN}_2(x_1))\right)$$

The MLP is two linear layers with a GELU in between, with `mlp_ratio = 4.0`.

When `window_size > 0`, `window_partition` cuts the feature map into windows. It pads the borders if H or W is not divisible by W_w. Attention runs separately in each window, then `window_unpartition` puts the tokens back in the full layout before the residual add. When `window_size = 0`, the block does global attention over all tokens.

I.4 Vision Transformer (SimpleViT)

Architecture

`SimpleViT` follows the plain ViT backbone from ViTDet (Li et al., ECCV 2022), adapted for MiniPlaces classification.

- Patch embedding: a 16×16 conv with stride 16 turns the 128×128×3 image into an 8×8 grid of 64 patch tokens, each of dimension $D = 192$.
- Positional embeddings: learnable 2D absolute embeddings of shape `(1, 8, 8, 192)`, initialized from a truncated normal ($\sigma = 0.02$) and added to the patch tokens.
- Blocks: 4 Transformer blocks with 4 heads each. Blocks 0 and 2 use window attention ($W_w = 4$) and blocks 1 and 3 use global attention. The window blocks pick up local features cheaply, and the global blocks let information move between windows.
- Stochastic depth: the DropPath rate goes linearly from 0.0 to 0.10 over the depth of the network.
- Head: LayerNorm, then global average pooling over the 8×8 tokens, then a linear layer from 192 dims to 100 scene classes.

Training

- Data: MiniPlaces, 128×128 crops, random horizontal flip, color jitter 0.15, random rotation ±15°, ImageNet normalization.
- Optimizer: AdamW, lr 1×10^{-3}, weight decay 0.05, batch size 512.
- Schedule: 1 linear warmup epoch, then cosine annealing over 60 epochs (61 epochs total).
- Precision: mixed precision (FP16 with GradScaler) on an NVIDIA Tesla T4. Training took about 2.5 hours.
- Weight decay is applied to linear and conv weights only. Norm weights, biases and positional embeddings are left out.

Results

Training loss went from 4.59 down to 2.17. Validation accuracy levelled off around epochs 55 to 60.

Metric	Result	Target / Baseline
Validation Top-1 Accuracy	40.58% (best: 40.65%)	~40.0%
Validation Top-5 Accuracy	70.40% (best: 70.51%)	-
Training Top-1 Accuracy	42.99%	-
Training Top-5 Accuracy	72.72%	-
Final Training Loss	2.1691	-

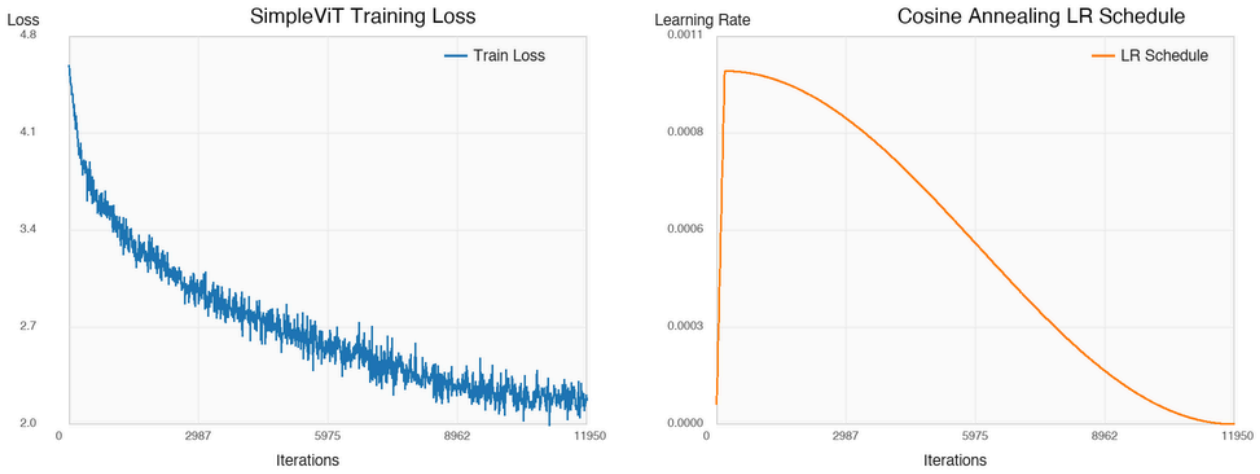


Figure 2: SimpleViT training loss and cosine annealing learning rate schedule across 61 epochs.

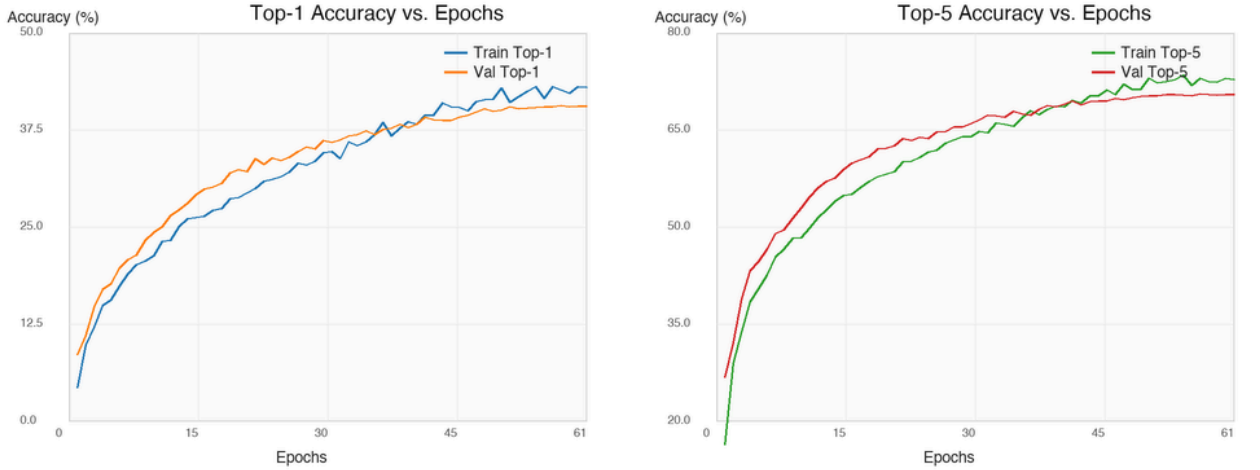


Figure 3: Training and validation accuracy for Top-1 (left) and Top-5 (right) across epochs.

SimpleViT reaches 40.58% top-1 and 70.40% top-5 on the MiniPlaces validation set. This is right around the ~40% baseline for a ViT trained from scratch. Training and validation accuracy are close (42.99% vs. 40.58%), so there is little overfitting. This is a small model (4 blocks, 192 dims) trained from scratch on a small dataset, so it is still well below the CNN and the pretrained ResNet-18.

Files: config `results/cls_vit/cls_miniplaces_vit.yaml` (and `config.txt`), logs `results/cls_vit/logs/`, summary `results/cls_vit/eval_summary.txt`, figures `results/cls_vit/figures/`, final checkpoint `results/cls_vit/model_final.pth.tar`.

Part II - Object Detection (FCOS)

We implemented FCOS, an anchor-free one-stage detector. The starter code already had the ResNet backbone, the FPN with extra P6/P7 levels, the shared heads, the point generator and the train/eval pipeline. We wrote the two missing parts in `det_libs/modeling.py`: inference and the training loss.

II.1 Inference

Inference runs on one image at a time. For each FPN level, the object score is `sqrt(sigmoid(cls) · sigmoid(center-ness))`. We drop candidates below `score_thresh = 0.1` and keep the top 1000 by score. The regression head predicts the distances `(l, t, r, b)` from the point to the four box sides, in grid units. We multiply them by the level stride and build each box as `(x−l, y−t, x+r, y+b)`. Boxes are clipped to the image and very small ones are removed. Then we merge all levels, run per-class NMS (`nms_thresh = 0.6`) and keep the top 100 detections. Class indices are shifted by `+1` to undo the label offset from data loading.

On the provided pretrained ResNet-18 model (VOC 2007 test):

Metric	Result
mAP@0.5	0.609
mAP@[0.5:0.95]	0.330
Inference time	~146 s

This matches the expected 60.9%, so the decoding is correct.

II.2 Training Loss

For each image we assign targets to every feature point, vectorized over points × GT boxes. A point is positive for a box if it lies inside a center-sampling window (`1.5 × stride`, clamped to the box) and the largest of its four distances is within that level's regression range. If a point matches several boxes, we take the one with the smallest area.

There are three losses, each divided by a running mean of the positive count:

- Classification: sigmoid focal loss (α=0.25, γ=2) over all points.
- Regression: GIoU over positive points, between the decoded and ground-truth boxes.
- Center-ness: BCE over positive points, with target `sqrt((min(l,r)/max(l,r))·(min(t,b)/max(t,b)))`.

Training settings (default config):

Backbone	ResNet-18, ImageNet-pretrained, BN frozen
Neck	FPN (256-d) + P6/P7, strides [8,16,32,64,128]
Input	scale jitter {256,288,320}, max side 480
Optimizer	SGD, momentum 0.9, weight decay 1e-4
LR	5e-3, cosine, 1 warmup epoch
Epochs / batch	10 (+1 warmup) / 4
GPU	single T4

Training took about 24 minutes. The total loss dropped from about 2.5 to 1.1. Classification and regression loss end near 0.25, and center-ness near 0.6.

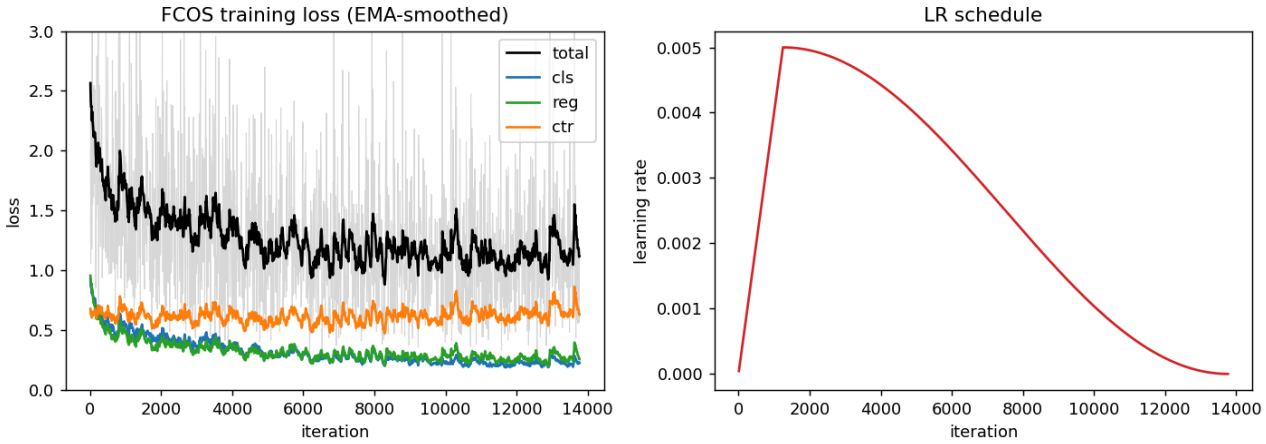


Figure 4: Training losses (total, classification, regression, center-ness) and the warmup + cosine learning-rate schedule.

Results on VOC 2007 test:

Metric	Ours	Pretrained
mAP@0.5	0.593	0.609
mAP@[0.5:0.95]	0.321	0.330
mAP@0.75	0.312	0.322
AP small/med/large	0.046 / 0.169 / 0.409	0.078 / 0.201 / 0.406

Our model gets 59.3% mAP@0.5, about 1.6 points below the pretrained reference. We think the gap is from our lighter schedule: batch 4, lr 5e-3, 10 epochs and 480-px input, compared to the reference's batch 32, lr 0.02, 12 epochs and 640-px input. Accuracy is best on large objects and worst on small ones, which makes sense at this input size.

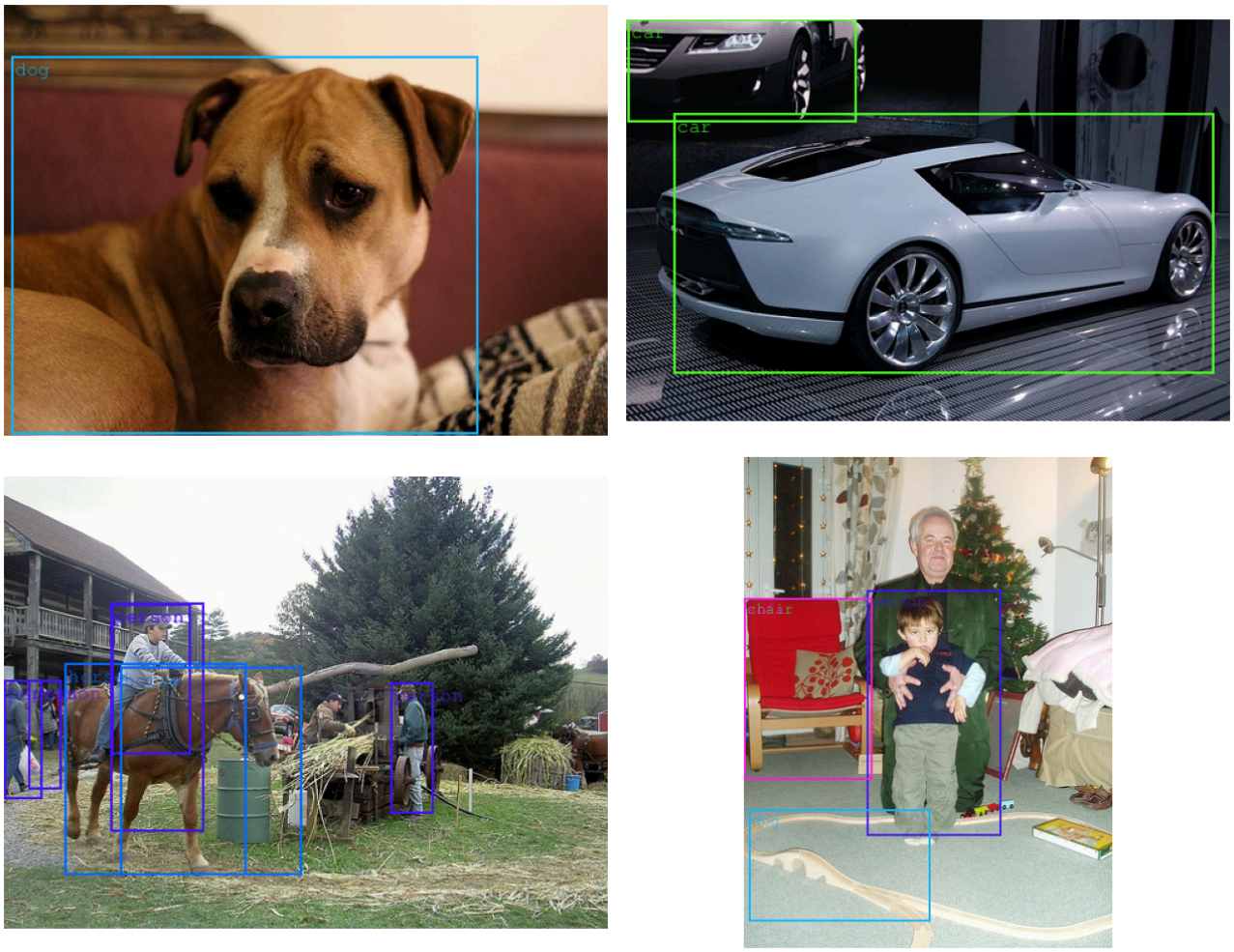


Figure 5: Sample detections on the test set, across different categories and scenes.

Contributions

- **Akshat**: Object detection (FCOS inference and training loss), repository/infra setup.
- **Ashvin**: Custom CNN design and training (`CustomNet`), fine-tuned ResNet-18 run, CHTC/Colab job setup, SimpleNet baseline run, CNN vs. SimpleNet vs. ResNet-18 comparison.
- **Shubhangi**: Multi-head self-attention (`Attention`), local-window Pre-LN Transformer block (`TransformerBlock`), and the Vision Transformer (`SimpleViT`): design, implementation, 60-epoch training on MiniPlaces, evaluation, and writeup.

Additional packages

Besides the required packages, we used `opencv-python-headless`, `matplotlib` (for plots), and `fonts-freefont-ttf` (so `eval_det.py -v` can draw label fonts on a headless server).

The ViT part needed nothing beyond the standard assignment packages (`torch`, `torchvision`, `numpy`, `opencv-python`, `pyyaml`, `pillow`, `tensorboard`, `gdown`, `pycocotools`).